



UNIVERSIDADE DE SÃO PAULO

Instituto de Ciências Matemáticas e Computação

SCC0504 - Programação Orientada a Objetos

RELATÓRIO DE DESENVOLVIMENTO DO JAVA CAFÉ

Sarah Meicy Machado Barbosa

Nº USP: 16830357

Larry Felype Silva Gonçalves

Nº USP: 16966460

São Carlos – SP

2026

REQUISITOS

Requisitos solicitados de acordo com o documento de projetos:

1. Interface do Usuário (User Interface)

- Interface Gráfica (GUI) com duas visualizações principais: Entrada de Pedidos (Order Entry) e Estoque (Inventory).
- Botões de itens do menu para a entrada de pedidos; o total acumulado deve estar sempre visível.

2. Processamento de Pedidos

- Adicionar e remover itens do menu; calcular automaticamente o subtotal, imposto e total.
- Finalizar uma venda (reduzindo o estoque) e exibir um recibo em texto em uma caixa de diálogo (dialog).

3. Gerenciamento de Estoque

- Lista de produtos contendo: nome, preço e quantidade em estoque.
- Diminuir o estoque automaticamente assim que um pedido for finalizado.
- Permitir que a equipe atualize os níveis de estoque.

4. Resumo de Vendas Diárias

- Botão para visualizar a receita total de hoje e o número de transações realizadas.

5. Persistência de Dados

- Salvar os registros de estoque e vendas em arquivos locais (CSV ou texto simples).
- Carregar os dados corretamente ao iniciar o sistema.

6. Tratamento de Exceções

- No mínimo uma exceção personalizada.
- Mensagens de erro amigáveis para o usuário em caso de entradas inválidas ou itens fora de estoque.

DESCRIÇÃO DO PROJETO

Nesta seção, descreveremos como o sistema desenvolvido implementa as funcionalidades propostas. Antes de fazê-lo, é necessário introduzi-los na estrutura geral da aplicação e suas principais classes.

O aplicativo aqui especificado foi desenvolvido com base na arquitetura model-view-controller. Começamos apresentando as classes que compõem a camada model, responsável por implementar a lógica de negócios:

- a) *Coffee*: esta classe representa um produto da loja e contém todos os atributos necessários para especificá-lo: nome, preços e quantidades em estoque.
- b) *Order*: esta classe representa um pedido feito pelo usuário. Cada pedido está associado a um determinado café e um determinado tamanho, de modo que só possa existir, ao mesmo tempo, um único objeto pedido por par (café, tamanho). Além desses atributos, o pedido também possui uma quantidade, preço unitário e preço total.
- c) *Inventory*: esta classe representa o estoque e seu único atributo é uma `ArrayList` contendo todos os produtos (*Coffee*) da loja. O principal propósito dessa classe é permitir o gerenciamento do estoque através de seus métodos.

A camada persistence é responsável por salvar e carregar os dados dos arquivos csv. Antes de abordarmos as classes que compõem esta camada, segue uma breve descrição dos arquivos csv manipulados por este projeto:

- a) *Sales.csv*: neste arquivo são registrados todos os pedidos feitos pelo usuário. Cada linha representa um pedido e contém os seguintes dados nesta ordem: quantidade, nome do café, tamanho, preço unitário, preço total.
- b) *Coffees.csv*: este arquivo guarda todos os produtos em estoque. Cada linha representa um produto e contém os seguintes dados nesta ordem: nome do café, ingredientes, as diferentes quantidades para as três opções de tamanho e os diferentes preços para cada tamanho.

Cada classe da camada persistence é responsável por lidar com a entrada e saída de cada um dos arquivos csv. As classes da camada persistence são:

- a) *InventoryCSVLoader*: É a classe responsável por lidar com o carregamento dos dados do inventário do arquivo csv para o programa.
- b) *InventoryCSVSaver*: É a classe responsável por lidar com o salvamento dos dados do inventário do programa para o arquivo csv quando solicitado.
- c) *SalesReportReader*: É a classe responsável por lidar com o carregamento dos dados das vendas do arquivo csv para o inventário. Pode carregar a quantidade de transações e o total da receita gerada.
- d) *SalesReportUpdater*: É a classe responsável por lidar com o registro de novas vendas no arquivo csv.

Depois temos a camada view, que compõe a interface gráfica de usuário. Ela é responsável por receber as ações e dados fornecidos pelo usuário. Dentre as principais classes da camada view encontram-se:

- a) *JavaCafeGUI (JFrame)*: esta classe cria a janela principal da aplicação e contém todos os demais componentes visuais. A janela foi dividida em duas áreas, uma para o cabeçalho, onde residem os botões para navegação entre as diferentes telas, e outra para o conteúdo, onde as diferentes telas ficam. Para o cabeçalho e para cada uma das duas telas (order entry screen e inventory screen) foram criadas classes específicas.
- b) *OrderEntryScreen (JPanel)*: esta classe agrupa todos os componentes necessários da tela de pedidos, são eles o menu e o carrinho.

- c) *Menu (JPanel)*: esta classe agrupa todos os produtos da loja e é a responsável por adicioná-los à interface.
- d) *ShoppingCart (JPanel)*: esta classe representa o carrinho de compras do usuário. Ela deve conter todos os pedidos criados e exibir os valores subtotal, taxa e total da soma de todos os pedidos no carrinho. Ela também contém botão para finalizar a compra.
- e) *Inventory Screen*: esta classe agrupa todos os componentes necessários da tela de estoque, sendo eles todos os produtos em estoque.

Ainda na camada view foram desenvolvidas outras classes, como as classes que representam os objetos coffee e order na GUI (CoffeeButton, CoffeeStockButton, OrderJPanel), e classes JDialog customizadas para input de dados do usuário e output de dados processados (PickSize, ReceiptJDialog, SaleSummaryDialog, UpdateStockDialog).

Já na camada controller, foram implementadas as classes responsáveis pelo processamento de dados, gerenciamento e coordenação geral da aplicação. São eles:

- a) *MenuController*: conecta a classe OrderEntryScreen (view) com a camada model formatando os atributos dos objetos coffee para que eles possam ser representados na camada view. O MenuController também acompanha as ações do usuário no menu, como quando o usuário tenta adicionar um pedido ao carrinho.
- b) *OrderController*: é responsável por criar e gerenciar os pedidos. Ele recebe e avalia as requisições do usuário para criar, remover, alterar e confirmar pedidos. Além de conectar a camada view com a classe Order, o OrderController também estabelece uma conexão com a classe InventoryController, para que os níveis de estoque sejam atualizados a cada compra.

- c) *InventoryController*: Esta classe estabelece uma conexão entre a classe *InventoryScreen* e a camada model recebendo e validando as requisições do usuário para alterar os níveis do estoque. Ela também exerce o papel de carregar todos os objetos *Coffee* na camada view, assim como a *MenuController* faz para a classe *OrderEntryScreen*.

Ainda na camada controller foram implementados dois Action Listeners, um que permite a navegação entre as diferentes telas e outro para exibir o *JDialog SaleSummaryDialog*.

Agora que foram devidamente apresentadas as principais classes que compõem a aplicação, podemos discorrer sobre como as funcionalidades do sistema foram implementadas.

I. Processamento de pedidos

O mecanismo de processamento de pedidos inicia-se na camada view no menu, onde o usuário realiza as seguintes operações:

- a) Usuário escolhe um café
- b) Usuário escolhe o tamanho do café
- c) O usuário tenta adicionar pedido ao carrinho

Na camada de controle, estas ações do usuário são acompanhadas pelo *Menu Controller*, que recebe a requisição do usuário e a repassa para o *Order Controller*. O *Order Controller* recebe a requisição e verifica se o produto está disponível no estoque. Se o produto não estiver disponível, o *Order Controller* lança a exceção *OutOfStockException*. O *Menu Controller* recebe essa exceção e a comunica para o usuário. Se o produto estiver disponível, sucedem-se as seguintes operações:

Se o pedido ainda não existe:

- a) [Order Controller] Cria um novo objeto pedido com as opções inseridas pelo usuário e adiciona-o ao array list de pedidos;
- b) [Order Controller] Formata os atributos do objeto pedido para que ele possa ser exibido na camada view;
- c) [Shopping Cart] Recebe dados formatados da camada controller e cria um componente gráfico para o pedido;
- d) [Order Controller] Recalcula os valores subtotal, taxa e total de todos os pedidos no carrinho;
- e) [Shopping Cart] Recebe os valores formatados e os exibe para o usuário.

Se o pedido (café, tamanho) já existe:

- a) [Order Controller] Tenta incrementar a quantidade do objeto pedido;
- b) [Order Controller] Recalcula os valores subtotal, taxa e total de todos os pedidos no carrinho;
- c) [Shopping Cart] Recebe os valores formatados e os exibe para o usuário.

Depois de criar um pedido e adicioná-lo ao carrinho, o usuário pode interagir com seus pedidos de três formas diferentes. São elas:

- a) *Alterar quantidade:* ao alterar a quantidade de um determinado pedido na camada view, o order controller recalcula os valores subtotal, taxa e total de todos os pedidos no carrinho e repassa os novos valores para a camada view. Não é necessário verificar se a nova quantidade é válida, pois na camada view o usuário é impedido de inserir um valor inválido devido à maneira como o componente JSpinner foi criado.
- b) *Remover pedido do carrinho:* o order controller recebe a requisição do usuário pela camada view e realiza as seguintes operações: Remove objeto order do array de pedidos; Comunica a camada view para que ela remova-o da interface também; Recalcula os valores subtotal, taxa e total de todos os pedidos no carrinho e repassa os novos valores para a camada view.

- c) *Finalizar compra*: o order controller comunica a compra ao Inventory Controller, que diminui o estoque e atualiza o arquivo csv do estoque; O order controller registra a venda no arquivo csv de vendas; O order controller remove todos os pedidos tanto da camada model quanto da interface; O order controller formata o texto do recibo com os dados de todos os pedidos no carrinho e exibe o recibo para o usuário.

II. Gerenciamento de estoque

O gerenciamento do estoque pode ser abordado de duas formas diferentes. A primeira refere-se a gestão automática por parte do sistema que deve ocorrer quando uma compra é realizada. Como fora mencionado anteriormente na descrição do processamento de pedidos, ao finalizar uma compra o order controller contata o inventory controller e sucedem-se as seguintes operações:

- a) [Inventory Controller] Recebe os atributos 'nome do café', 'tamanho' e 'quantidade' do pedido;
- b) [Inventory Controller] Utiliza método do objeto inventory para diminuir a quantidade em estoque do café especificado;
- c) [Inventory Screen] Recebe dados formatados pelo inventory controller e atualiza os valores em estoque na interface gráfica.
- d) [Inventory Controller] Utiliza método da camada de persistência para salvar novos valores do estoque no arquivo csv.

A segunda forma de abordar o gerenciamento de estoque diz respeito à tentativa do usuário de atualizar os valores em estoque de cada produto. Neste

caso, o inventory controller é responsável por receber e validar essas requisições do usuário e realizar as seguintes operações:

- a) [Inventory Controller] Recebe dados de entrada fornecidos pelo usuário pela camada view.
- b) [Inventory Controller] Verifica se os valores são válidos
- c) [Inventory Controller] Utiliza método do objeto inventory para atualizar os valores em estoque de determinado café.
- d) [Inventory Controller] Utiliza método da camada de persistência para salvar novos valores do estoque no arquivo csv.
- e) [Inventory Controller] Avisa ao Order Controller qual café teve o estoque alterado.
- f) [Order Controller] Verifica se o café com estoque alterado está no carrinho, e atualiza a quantidade máxima que é possível comprar desse café.

III. Resumo de vendas diárias

Para iniciar a rotina de exibição de resumo de vendas do dia, o usuário precisa clicar no botão “Sales Summary” exibido no header, e então o programa seguirá as seguintes etapas:

- a) [Sales Summary Listener] O listener recebe a informação e abre a janela de informações.
- b) [Sales Summary Dialog] Chama Sales Report Reader para ler o total de transações e de receita no dia.
- c) Exibe o total de transações e de receita.

IV. Persistência de dados

O programa precisa salvar e carregar dados em momentos específicos do programa, sendo esses:

- Ao iniciar o programa:

a) O programa instancia um objeto inventário e carrega os dados de “coffee.csv” usando a classe Inventory CSV Loader.

- Ao atualizar o inventário:

a) O programa recebe o café cujo inventário será atualizado

b) Guarda o valor novo no objeto inventário

c) Salva o inventário no arquivo CSV usando o método Inventory CSV Saver

- Ao finalizar um pedido

a) Encontra o café no inventário e decrementa a quantidade comprada

b) Salva o inventário no arquivo CSV usando o método Inventory CSV Saver

c) Para cada pedido, salva os dados do pedido no final do arquivo “sales.csv”

PROBLEMAS

Nesta seção, abordaremos problemas que surgiram durante o desenvolvimento do projeto e como lidamos com eles.

I. Caminho relativo para os arquivos csv

Durante o processo de polimento do projeto, decidimos criar uma pasta dedicada aos arquivos persistentes do projeto. Ao fazer isso, encontramos o problema de que o código estava acessando os arquivos por um caminho relativo, que esperava que a aplicação fosse executada a partir da pasta src.

Solução: Os caminhos foram revisados para considerar a pasta raiz do projeto como diretório de execução.

II. Atualização parcial de estoque

Ao atualizar o estoque, era esperado que os três valores do estoque dos três tamanhos de café fossem inseridos corretamente, caso contrário, a exceção `NumberFormatException` era lançada.

Solução: Para tornar a interface mais amigável ao usuário, agora ao atualizar o estoque a verificação passa a ser individual para cada novo valor de estoque de cada um dos tamanhos, permitindo que seja atualizado quantos tamanhos forem necessários, e não todos juntos.

III. Comunicação entre model, view e controller

Após atualizar o estoque no modelo **Inventory**, os valores na tela permaneciam inalterados, pois a view não era notificada das alterações.

Solução: Foi criado um método para localizar e atualizar o componente visual relacionado ao café alterado no inventário.

IV. Organização do processo de compilação

A compilação manual do projeto, especialmente após a implementação da biblioteca FlatLaf (Look and Feel) precisava de comandos grandes e repetitivos, ocasionalmente gerando alguns erros durante o processo de compilação.

Solução: Foi desenvolvido um script “run” responsável por limpar compilações antigas, compilar todas as classes, copiar os recursos necessários e iniciar a aplicação.

V. Layout responsivo das telas

Conforme mais produtos eram adicionados ao inventário, os componentes da interface deixavam de caber na tela, prejudicando a visualização.

Solução: Foi substituído o layout padrão por um **WrapLayout**, permitindo que os componentes fossem automaticamente reorganizados em novas linhas quando mais espaço fosse necessário, e exibindo um scroll caso a tela não possuísse espaço suficiente.

PLANO DE TESTES

A aplicação foi testada manualmente para verificar o funcionamento correto das principais funcionalidades do sistema e da interface gráfica.

I. Gerenciamento de Estoque

- Atualização de estoque com valores válidos;
- Atualização de estoque com valores inválidos;
- Atualização parcial do estoque, alterando apenas alguns tamanhos
- Atualização com campos vazios;
- Cancelamento e confirmação da operação;
- Alteração de estoque que está sendo utilizado como pedido no carrinho.

II. Gerenciamento de pedidos

- Adição de produtos ao carrinho.
- Adição de múltiplas unidades de um mesmo produto.
- Remoção de itens do carrinho.
- Finalização de compras.
- Tentativa de compra de quantidades superiores ao estoque disponível.

III. Persistência de dados

- Salvamento das alterações em estoque no arquivo “coffees.csv”;

- Registro das vendas no arquivo “sales.csv”;
- Carregamento correto dos dados ao iniciar a aplicação.

IV. Interface Gráfica

- Navegação entre as telas do sistema;
- Exibição correta dos componentes visuais;
- Funcionamento dos scrolls nas telas de menu e inventário;
- Exibição das mensagens de erro e janelas de operação.

Resultados: Todos os testes demonstraram que as funcionalidades operam de acordo com o esperado, e todos os problemas citados neste relatório foram solucionados.

PROCEDIMENTO DE COMPILAÇÃO/EXECUÇÃO

Foi desenvolvido um script run para facilitar a execução do programa. Ele foi disponibilizado junto ao projeto no GitHub.

- O usuário precisa ter o Java Development Kit 8+ ou superior para executar o programa.
- O usuário deve baixar o repositório e executar o script run, usando “[run.sh](#)” caso esteja usando um sistema LINUX e “run.bat” caso esteja usando um sistema Windows.
- Caso esteja utilizando um sistema LINUX é preciso antes dar permissão de execução para o arquivo run, com o comando: **chmod +x run.sh**